

Тестирование веб-сайта или веб-приложения

Включает в себя 10 основных этапов:

- 1) Documentation Testing (тестирование документации);
- 2) Functionality Testing (функциональное тестирование);
- 3) GUI Testing (тестирование графического интерфейса/вёрстки);
- 4) Usability Testing (тестирование удобства интерфейса);
- 5) Interface Testing (тестирование интерфейса);
- 6) Database Testing (тестирование БД);
- 7) Compatibility Testing (тестирование совместимости);
- 8) Performance Testing (тестирование производительности);
- 9) Security Testing (тестирование безопасности);
- 10) Crowd Testing (краудсорсинговое тестирование).

Рассмотрим их более подробно.

1 Documentation Testing

Плохая документация может повлиять на качество продукта. Хорошая документация по продукту играет решающую роль в конечном продукте. Таким образом, тестирование документации играет жизненно важную роль в тестировании программного обеспечения. Тестирование задокументированных артефактов, разработанных до, во время и после тестирования продукта, называется тестированием документации.

Ниже приведены некоторые часто используемые артефакты:

- Requirement documents;
- Test Plan;
- Test Cases;
- Traceability Matrix (RTM).

Подробно о тестировании документации написано в видах тестирования.

2 Functionality Testing

Функциональное тестирование – тестирование того, что делает система. Проверить, что каждая функция программного приложения ведет себя так, как

указано в документе с требованиями. Тестирование всех функций путем предоставления соответствующих входных данных, чтобы проверить, соответствует ли фактический результат ожидаемому результату. Оно используется для проверки рабочих процессов, всех ссылок веб-страниц, тестирования форм, тестирования файлов cookie и подключения к базе данных. Обычно функциональное тестирование включает в себя следующие задачи:

- **Testing UI Workflows:** тестируются end to end workflow или бизнес-сценарии. Рекомендуется написание тестовых сценариев или тестовых случаев, чтобы охватить различные сценарии и установить критерии прохождения;

- **Тестирование гиперссылок:** все ссылки на веб-сайте работают правильно, и нет неработающих ссылок. Типы ссылок включают внутренние ссылки, исходящие ссылки, якорные ссылки, схемы mailto и т. д.;

- **Тестирование форм** (проверка полей ввода): формы используются для интерактивного общения с конечными пользователями. Тестировщик должен убедиться, что все формы работают должным образом. Тестирование форм включает в себя:

- заполняются ли значения по умолчанию;
- отображается ли сообщение об ошибке, когда пользователь не заполняет обязательное поле;
- принимает ли форма недопустимые значения;
- формы оптимально отформатированы для лучшей читабельности;
- поля AJAX правильно заполняют значения во время выполнения;
- загружаются ли раскрывающиеся списки с параметрами.

- **Проверка файлов cookie:** подробно о тестировании кук написано в теме про cookie в сетях;

- **Проверка HTML и CSS:** Тестировщик должен проверить, имеет ли сайт чистую структуру HTML и оптимизированный CSS в соответствии со

стандартами W3C. Также нужно убедиться, что поисковые системы могут легко сканировать сайт.

- нет синтаксических ошибок HTML;
- цветовые схемы читаемы;
- карта сайта точна.

Примеры функциональных тест-кейсов:

• Кнопки:

- Enter должна срабатывать как submit;
- Tab должен переводить курсор на следующий элемент.

• Поля ввода:

- trimming («убирание») пробелов в полях ввода;
- пустота/пробелы в поле ввода;
- все способы редактирования (Insert, Delete, Backspace, Ctrl+C/V/X/Z и т. д.);
- дроби (1.5/ 1,5/ 1/5).

• Поиск:

- wildcard symbols (*, вертикальный слеш, ?);
- написание поискового запроса слитно/раздельно/через дефис должно вести к одному результату;
- ввод текста в другой раскладке.

• Сообщения об ошибках:

- пробуем отключить в настройках браузера.

• Календарь:

- 31 июня;
- 29 февраля + не високосный год;
- прошлое/будущее (например, купить билет на уже прошедшее число).

• Время:

- синхронизация с сервером (на сервере приложения может быть выставлено другое время, отличающееся от таймзоны пользователя);
- временные зоны.
- E-mail:
 - логин (63 символа) @ домен (253 символа (может быть ip)).
- Всплывающие окна / подсказки:
 - пробуем закрыть разными способами (нажатие на кнопку (если есть), на «крестик», клавишей ESC, просто нажатием в другую область экрана);
 - рефреш страницы особенно в момент запроса на сервер (например, совершение транзакции по покупке) иногда может приводить к появлению ошибок.
- Все обязательные поля должны быть валидированы.
- Звездочка должна отображаться для всех обязательных полей.
- Не должно отображаться сообщение об ошибке для дополнительных полей.
- Числовые поля не должны принимать буквы и должно отображаться соответствующее сообщение об ошибке.
- Проверьте наличие отрицательных чисел, если это разрешено для числовых полей.
- Тестовое деление на ноль должно быть правильно обработано.
- Проверьте максимальную длину каждого поля, чтобы убедиться, что данные не усекаются.
- Текст всплывающего сообщения («Это поле ограничено 500 символами») должен отображаться, если данные достигают максимального размера поля.
- Проверьте, должно ли отображаться подтверждающее сообщение для операций обновления и удаления.
- Величины должны быть в подходящем формате.

- Проверьте все поля ввода на ввод специальных символов.
 - Проверьте функциональность тайм-аута.
 - Проверьте функциональность сортировок.
 - Проверьте, что FAQ и Политика конфиденциальности четко определены и доступны для пользователей.
 - Проверьте, все ли работает и не перенаправляется ли пользователь на страницу ошибки.
 - Все загруженные документы открываются правильно.
 - Пользователь должен иметь возможность скачать загруженные файлы.
 - Проверьте функциональность электронной почты системы.
- Тестируемый скрипт корректно работает в разных браузерах (IE, Firefox, Chrome, Safari и Opera).
- Проверьте, что произойдет, если пользователь удалит файлы cookie, находясь на сайте.
 - Проверьте, что произойдет, если пользователь удалит файлы cookie после посещения сайта.
 - Проверка работоспособности при наличии расширений браузера, например, блокировщиков рекламы.

3 GUI Testing

Верстка – размещение элементов веб-приложения (изображения, текст, кнопки, видео...) в соответствии с макетом или требованиями.

Проверяем:

- наличие всех элементов;
- их размер и цвет;
- расположение относительно друг-друга.
- Сравнение с макетом – метод наложения готового эталонного макета (обычно psd-файл) на приложение в экране браузера, все несовпадения можно рассматривать как ошибки (для этого есть хороший инструмент [Pixel Perfect](#)).

- Измерение размеров элемента – если это имеет значение, то померять размеры элемента и сравнить их со спецификацией можно с помощью, например [Page Ruler](#).

- Правильность шрифтов (название, размер, цвет) – [WhatFont](#).

- Цвета интерфейса – [ColorZilla](#).

- Контент – проверить на наличие орфографических и грамматических ошибок ([SpellChecker](#)).

- Появление курсора - довольно часто мы забываем проверить, появляется ли вообще и как выглядит курсор в полях ввода, на кликабельных элементах.

- Фавикон – такая маленькая незначительная вещица, но может изрядно подпортить впечатление пользователя (в моей практике были случаи, когда разработчики или дизайнеры шаблона оставляли фавикон с логотипом своей компании на сайте у заказчика).

- Обозначение возможности переноса элементов.

- Кодировка (UTF8...).

- Стандарты HTML/CSS – достаточно неплохие решения для быстрой проверки предлагает [W3C](#).

- Заголовки по всему приложению должны быть приведены к одному стандарту.

- Title страницы – о нём мы тоже часто забываем, также как и разработчики :)

- Back button – достаточно часто встречается ошибка при переходе на какую-то страницу и нажатии на браузерную кнопку Back, предыдущая страница крашится или возврат на нее вовсе не осуществляется.

- Масштабируемость – особенно это важно при тестировании на смартфонах и планшетах. Где пользователь часто меняет масштаб экрана ([Window Resizer](#)), а также режим адаптивного дизайна (например в FireFox Developer Edition).

- Кроссбраузерность – одна и та же страница может выглядеть по-разному в разных браузерах.

- Проверяем Scroll.
- Браузерные расширения, которые могут влиять на внешний вид приложения (например, Adblock) – пробуем включить и отключить.
- Проверить контент при отключенных (режим WebDeveloper) изображениях, flash, JavaScript.

Локализация – что мы знаем об этом? Обычно наши знания сводятся к невятным «ну, это язык», «кодировка», «раскладка», еще реже «геолокация». Что еще мы так часто забываем проверять в рамках тестирования локализации?

- Проверяем тестовый образец на правильность перевода - тут, конечно, хорошо бы подключить переводчика или носителя языка, но за неимением таких, берем тестовый образец и переводим через любой онлайн-переводчик (ну и все мы помним, как прекрасно и весело читать описание товаров на русском языке на AliExpress).

- Длина переведенных слов – количество символов в переведенном слове может быть гораздо больше, что может привести к «расползанию» интерфейса при переводе.

- Сокращения/аббревиатуры – существуют правила, по которым их либо переводят, либо транслитерируют, либо оставляют как есть.

- Валюта.

- Параметры шрифта могут также значительно отличаться в зависимости от языка ввода.

- Проверить работу поиска во всех локализациях – например, на AliExpress результаты поиска одного и того же слова «смартфон» дают разный результат по количеству найденных товаров, причем разница исчисляется десятками тысяч.

- Мета-информация (keywords/title/description) – столь незначительное для пользователя, невидимое, но такое важное для поисковых машин и продвижения сайта в гугле и других поисковиках.

- RTL (right to left languages) – языки с обратным написанием (арабский, иврит) имеют свои особенности: числа пишутся слева направо, значки и иконки отзеркаливаются, названия программ не переводятся, нет переносов, кнопки редактирования Backspace и Delete работают наоборот.

4 Usability Testing

Юзабилити-тестирование стало важной частью любого веб-проекта. Его могут провести тестировщики или небольшая фокус-группа, похожая на целевую аудиторию веб-приложения чтобы проверить, является ли приложение удобным для пользователя, и было ли комфортно использовать его конечному пользователю:

- Соответствует ли приложение ожиданиям конечного пользователя;
- Логичность интерфейса;
- Самое нужное «сверху»;
- Продуманная навигация;
- Локализация (да, да, она относится и сюда тоже);
- Совместимость с другим софтом (соцсети) и железом;
- Скорость работы приложения;
- Информативность (сообщения / обязательные поля);
- Возможность отмены действий пользователя;
- Help - должна быть инструкция, как работать с приложением;
- Возможность печати (если нужно).

Примеры юзабилити тест-кейсов:

- Текст подсказки должен быть там для каждого поля.
- Домашняя ссылка должна быть на каждой странице.
- Сообщение о подтверждении должно отображаться для любого вида операции обновления и удаления.
- Полоса прокрутки должна появляться только при необходимости.
- Если при отправке появляется сообщение об ошибке, информация, заполненная пользователем, должна быть там.
- Название должно отображаться на каждой веб-странице.

- Все поля (текстовое поле, раскрывающийся список, переключатель и т.д.) и кнопки должны быть доступны с помощью сочетаний клавиш, и пользователь должен иметь возможность выполнять все операции с помощью клавиатуры.

5 Interface Testing

Тестирование интерфейсов предназначено для проверки интерфейса между веб-сервером и сервером приложений, правильно ли взаимодействуют сервер приложений и сервер баз данных. Это гарантирует положительный пользовательский опыт. Он включает в себя проверку процессов связи, а также проверку правильности отображения сообщений об ошибках.

- Приложение: тестовые запросы правильно отправляются в базу данных и вывод на стороне клиента отображается правильно. Ошибки, если таковые имеются, должны быть обнаружены приложением и должны отображаться только администратору, а не конечному пользователю;

- Веб-сервер: тестовый веб-сервер обрабатывает все запросы приложений без какого-либо отказа в обслуживании;

- Сервер базы данных: запросы, отправленные в базу данных, дают ожидаемые результаты. Проверьте реакцию системы, когда невозможно установить соединение между тремя уровнями (Приложение, Интернет и База данных) и соответствующее сообщение отображается конечному пользователю.

6 Database Testing

Тестирование баз данных (back-end тестирование, тестирование данных) включает проверку целостности данных на front end с данными на back end. Оно проверяет схему, таблицы базы данных, столбцы, индексы, хранимые процедуры, триггеры, дублирование данных, потерянные записи, ненужные записи. Оно включает в себя обновление записей в базе данных и их проверку на внешнем интерфейсе.

Тестирование будет включать в себя:

- Отображение ошибок при выполнении запросов;

- Целостность данных поддерживается при создании, обновлении или удалении данных в базе данных;

- Тестирование производительности базы данных;
- Тестирование процедур, триггеров и функций.

Примеры тест-кейсов для тестирования базы данных:

- Проверьте имя базы данных: имя базы данных должно соответствовать спецификациям.

- Проверьте таблицы, столбцы, типы столбцов и значения по умолчанию: все должно соответствовать спецификациям.

- Проверьте, допускает ли столбец null значение.
- Проверьте первичный и внешний ключ каждой таблицы.
- Проверьте, установлена ли сохраненная процедура или нет.
- Проверьте имя хранимой процедуры
- Проверьте имена параметров, типы и количество параметров.
- Проверьте требуемые параметры.
- Проверьте хранимую процедуру, удалив некоторые параметры
- Проверьте, когда выход равен нулю, это должно повлиять на нулевые записи.

- Проверьте хранимую процедуру, написав простые запросы SQL.
- Проверьте, возвращает ли хранимая процедура значения
- Проверьте хранимую процедуру с образцами входных данных.
- Проверьте поведение каждого флага в таблице.
- Убедитесь, что данные правильно сохраняются в базе данных после каждой отправки страницы.

- Проверьте данные, если выполняются операции DML (Обновить, удалить и вставить).

- Проверьте длину каждого поля: длина поля на Frontend и backend должна быть одинаковой.

- Проверьте имена баз данных QA, UAT и production. Имена должны быть уникальными.

- Проверьте зашифрованные данные в базе данных.
- Проверьте размер базы данных.
- Также проверьте время ответа каждого выполненного запроса.
- Проверьте данные, отображаемые на Frontend, и убедитесь, что они совпадают с backend.
- Проверьте достоверность данных, вставив неверные данные в базу данных.
- Проверьте триггеры.

7 Compatibility Testing

Тестирование совместимости предназначено для проверки совместимости приложения в разных браузерах и на различных устройствах.

- Тестирование совместимости браузера: кросс-браузерное тестирование – это тип нефункционального теста, который помогает нам убедиться, что наш веб-сайт или веб-приложение работают должным образом в различных веб-браузерах. При тестировании веб-сайта нам необходимо убедиться, что он отображается одинаково во всех браузерах. Нам нужно предоставить одинаковый опыт для пользователей, независимо от того, какой тип ОС и какой браузер они используют. Не все используют одну и ту же среду. Несмотря на то, что Google Chrome является самым популярным на текущем рынке, все же множество пользователей используют Mozilla Firefox, Safari и другие. Если веб-сайт не работает должным образом в конкретном браузере, это ухудшает взаимодействие с пользователем. Нужно проверить, правильно ли отображается ваше веб-приложение в браузерах, работает ли JavaScript, AJAX и аутентификация. Вы также можете проверить рендеринг веб-элементов, таких как кнопки, текстовые поля и т. д.

- Тестирование совместимости устройств: этот тест подтверждает, что веб-приложение responsive и работает на устройствах разного размера и с разными операционными системами.

Примеры тестов на совместимость:

- Протестируйте сайт в разных браузерах (IE, Firefox, Chrome, Safari и Opera) и убедитесь, что сайт отображается правильно.
- Используемая версия HTML совместима с соответствующими версиями браузера.
- Проверьте правильность отображения изображений в разных браузерах.
- Протестируйте шрифты, которые можно использовать в разных браузерах.
- Протестируйте код Javascript в разных браузерах.
- Проверьте анимированные GIF-файлы в разных браузерах.

8 Performance Testing

Тестирование производительности определяет или подтверждает характеристики скорости, масштабируемости и/или стабильности тестируемой системы или приложения. Производительность связана с достижением времени отклика, пропускной способности и уровня использования ресурсов, которые соответствуют целям производительности для проекта или продукта. Тестирование производительности веб-приложений проводится для снижения риска доступности, надежности, масштабируемости, быстродействия, стабильности и т. д. системы. Тестирование производительности включает в себя ряд различных типов тестирования, таких как нагрузочное тестирование, объемное тестирование, стресс-тестирование, тестирование емкости, тестирование выдержки/выносливости и пиковое тестирование, каждое из которых предназначено для выявления или решения проблем с производительностью в системе.

Примеры тестов:

- Имитируем нагрузку пользователями (JMeter);
- Пробуем загрузить большие объемы данных, файлы, медиа;
- Нагружаем БД;
- Понижаем скорость инета (NetLimiter);
- Понижаем скорость передачи данных (Throttling);

- Тестируем восстановление системы после падений.

9 Security Testing

Тестирование безопасности – это процесс, позволяющий определить, защищает ли система данные и поддерживает ли она функциональность, как предполагалось. Тестирование безопасности направлено на выявление всех возможных лазеек и слабых мест системы на самом начальном этапе, чтобы избежать нестабильной работы системы, неожиданного сбоя, потери информации, потери дохода, потери доверия клиентов. Тесты безопасности включают тестирование на наличие уязвимостей, таких как:

- SQL-инъекция (SQL Injection);
- Межсайтовый скриптинг (XSS);
- Управление сеансом (Session Management);
- Сломанная аутентификация;
- Подделка межсайтовых запросов (CSRF);
- Неправильная конфигурация безопасности;
- Невозможность ограничить доступ к URL-адресу;
- Раскрытие секретных данных;
- небезопасная прямая ссылка на объект;
- Отсутствует контроль доступа на функциональном уровне;
- Использование компонентов с известными уязвимостями;
- Непроверенные перенаправления и возвраты.

Примеры тестовых сценариев для тестирования безопасности:

- веб-страница, содержащая важные данные, такие как пароль, номера кредитных карт, секретные ответы на секретный вопрос и т.д. должна быть отправлена через HTTPS (SSL).

- важная информация, такая как пароль, номера кредитных карт и т.д. должна отображаться в зашифрованном виде.

- правила проверки пароля применяются на всех страницах аутентификации, таких как Регистрация, забытый пароль, смена пароля.

- если пароль изменен, пользователь не должен иметь возможность войти со старым паролем.

- сообщения об ошибках не должны отображать важную информацию.

- если пользователь вышел из системы или сеанс пользователя истек, пользователь не должен перемещаться по сайту авторизованным.

- проверьте доступ к защищенным и незащищенным веб-страницам напрямую без входа в систему.

- опция «Просмотр исходного кода» отключена и не должна быть видна пользователю.

- учетная запись пользователя заблокирована, если пользователь вводит неправильный пароль несколько раз.

- куки не должны хранить пароли.

- если какая-либо функция не работает, система не должна отображать информацию о приложении, сервере или базе данных. Вместо этого она должна отображать пользовательскую страницу ошибки.

- проверьте атаки SQL-инъекций.

- проверьте роли пользователей и их права. Например, запрашивающая сторона не должна иметь доступа к странице администратора.

- важные операции записаны в файлы журналов, и эта информация должна быть отслеживаемой.

- значения сеанса находятся в зашифрованном формате в адресной строке.

- информация о файлах cookie хранится в зашифрованном формате.

- проверьте приложение на брутфорс-атаки

10 Crowd Testing or Crowdsourced testing

Краутестинг или краудсорсинговое тестирование – это новая тенденция в тестировании программного обеспечения, которая использует толпу (crowd/большое количество людей) для выполнения тестов, которые в противном случае были бы выполнены выбранной группой людей в компании. Краудсорсинговое тестирование представляет собой интересную и перспективную концепцию и помогает выявить многие незамеченные

дефекты. Оно включает в себя практически все типы тестирования, применимые к вашему веб-приложению.

Список использованных источников

1	Тестирование	веб-сайта	или	веб-приложения	—
https://vladislavermeev.gitbook.io/qa_bible/testirovanie-v-raznykh-sferakh-oblastyakh-testing-different-domains/testirovanie-veb-saita-ili-veb-prilozheniya-web-application					
2	Чек-лист	тестирования	WEB	приложений	—
https://habr.com/ru/articles/542422					